

WE LIKE TO
SHOP



Your Virtual Journey Starts Here
★★★★★ \$80.20

[View details](#)



Weird Crushes Game
★★★★★ \$22.40

[View details](#)



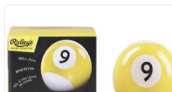
Babbage Web Spray
★★★★★ \$65.44

[View details](#)



All-in-One Typewriter
★★★★★ \$95.45

[View details](#)



Actividad 10

File path traversal, traversal sequences stripped with superfluous URL-decode

Guerrero Hurtado Lizbeth

Marco Teórico

¿Qué es File Path Traversal?

Es una vulnerabilidad de control de acceso que ocurre cuando una aplicación web no valida ni sanitiza los parámetros provistos por el usuario al construir rutas del sistema de archivos. Esto permite a un atacante salir del directorio público (webroot) mediante secuencias de escape y acceder a archivos sensibles alojados en el servidor.

Traversal Sequences Stripped with Superfluous URL-Decode:

Este escenario ocurre cuando la aplicación intenta defenderse inspeccionando o sanitizando la entrada en busca de secuencias como `../` en texto plano o con una primera capa de codificación. Sin embargo, antes de pasar el parámetro a la función que lee el archivo en el sistema operativo, la aplicación (o el framework subyacente) ejecuta un proceso adicional e innecesario de decodificación URL (superfluous URL-decode).

Mecanismo de Explotación:

Si el servidor aplica una decodificación secundaria de forma automática, el atacante puede codificar doblemente (double URL encode) los caracteres críticos del payload. Al enviar los caracteres % codificados como %25, la secuencia `../` se transforma en `%252e%252e%252f`:

Payload enviado: `%252e%252e%252f` → Decodificación URL 1(Servidor/WAF): `%2e%2e%2f`

Sanitización (Busca “`../`” en texto plano): Pasa la validación porque `%2e%2e%2f` ≠ `../`

Decodificación URL 2 (Superfluous decode): `%2e%2e%2f` → Resultado final: `../`

Punto Clave de Seguridad:

El orden del procesamiento de los datos es crítico. Decodificar la entrada del usuario después de haber realizado las validaciones o filtros de seguridad invalida por completo cualquier control previo. La decodificación y normalización de la entrada siempre deben ejecutarse antes de aplicar las reglas de validación.

Impacto en la Tríada CIA:

Confidencialidad (Alto): Compromiso crítico del pilar. Un atacante puede acceder a archivos del

sistema de archivos como /etc/passwd, archivos con credenciales de base de datos (.env, config.php), claves privadas SSH y código fuente.

Integridad (Bajo): En este reto el vector es exclusivamente de lectura, aunque las credenciales obtenidas podrían ser usadas posteriormente para modificar información.

Disponibilidad (Medio): La lectura repetida de archivos pesados o dispositivos especiales (/dev/urandom) mediante este bypass puede derivar en una Denegación de Servicio (DoS).

Procedimiento de Explotación (Metodología en Burp Suite)

Reconocimiento y Captura de Petición:

- Abrir el navegador integrado de Burp Suite y acceder a la aplicación del laboratorio.
- Seleccionar cualquier producto de la tienda virtual para forzar la carga de su imagen correspondiente.
- Ir a la pestaña Proxy > HTTP history y ubicar la solicitud de carga de imagen: GET /image?filename=42.jpg HTTP/1.1

Envío al Módulo Repeater:

- Hacer clic derecho sobre la petición y seleccionar Send to Repeater (o presionar Ctrl + R).

Prueba de Payloads Estándar (Comprobación del Bloqueo):

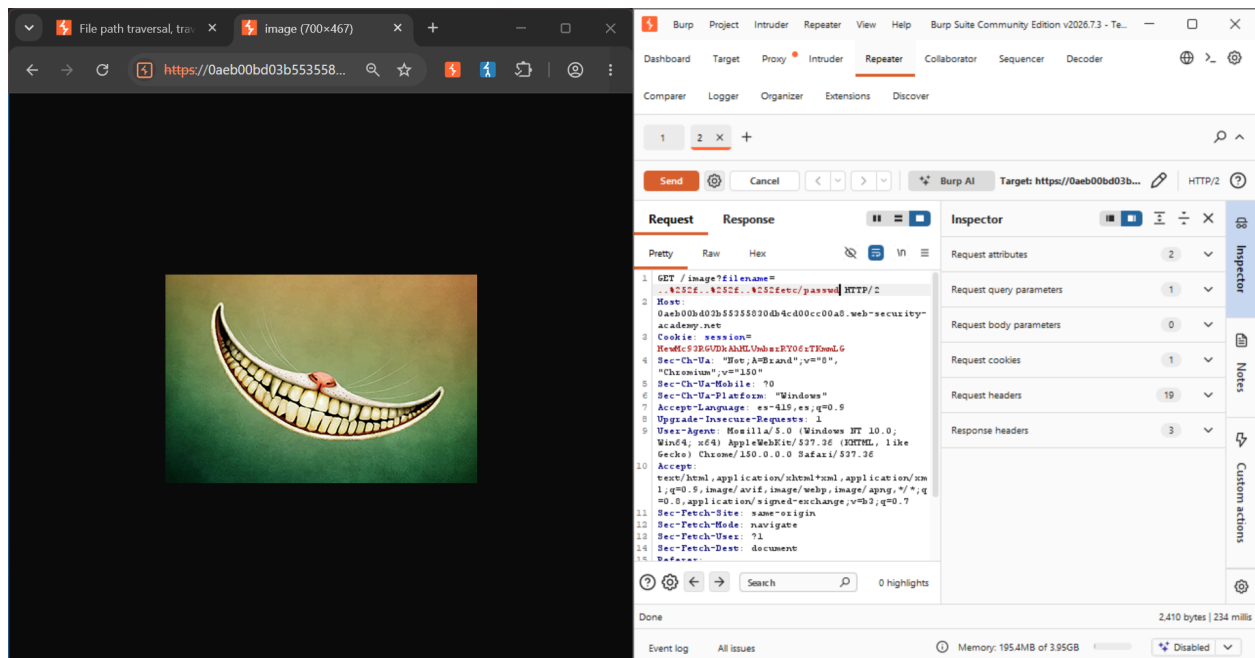
- En Repeater, probar enviando ?filename=../../../../etc/passwd o la versión con codificación URL simple ?filename=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd.
- Enviar la petición y observar que la aplicación bloquea la solicitud o no encuentra el archivo, confirmando la presencia de un filtro sobre la entrada en primera capa.

Inyección del Payload con Doble Codificación URL:

- Reemplazar los puntos (.) y barras (/) por sus equivalentes con doble codificación URL.:
 - . (Punto) → %2e --> %252e
 - / (Barra) → %2f → %252f
- Modificar el parámetro filename en Repeater con el siguiente formato: GET /image?filename=%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd HTTP/1.1

Exfiltración y Confirmación:

- Hacer clic en Send.
- Verificar que la respuesta del servidor sea un código HTTP/2 200 OK.
- Confirmar en el cuerpo de la respuesta (Response Body) el contenido del archivo /etc/passwd.
- Refrescar la página en el navegador para verificar la actualización del banner a "Lab Solved".



Estrategias de Mitigación Recomendadas

Normalizar la Entrada Antes de Validar:

Si es inevitable procesar codificaciones URL en el backend, el sistema debe realizar toda la decodificación y canonización de la entrada antes de pasar la cadena por cualquier filtro de seguridad o expresión regular.

Sanitización Mediante Listas Blancas (Whitelisting):

Descartar las listas negras. Validar que la entrada cumpla estrictamente con un formato de caracteres alfanuméricos y una extensión permitida mediante expresiones regulares:

```
^[a-zA-Z0-9_-]+\.(jpg|png|gif)$
```

Verificación de Ruta Canónica (Canonicalization Check):

Resolver la ruta canónica absoluta en el servidor y confirmar de forma programática que empiece dentro de la carpeta destinada a las imágenes.

Ejemplo en Java:

```
// Supongamos que el framework ya realizó la decodificación URL previa

File file = new File(BASE_DIRECTORY, userInput);

String canonicalPath = file.getCanonicalPath();

if (!canonicalPath.startsWith(BASE_DIRECTORY)) {

    throw new SecurityException("Intento de Path Traversal por doble codificación detectado.");

}
```

Principio de Menor Privilegio:

Ejecutar el servicio web bajo un usuario del sistema operativo con permisos fuertemente restringidos (ej. www-data), garantizando que no tenga acceso de lectura a /etc/passwd u otros archivos del sistema.

Conclusiones y Aprendizaje

- **Fallos en el Orden de Decodificación:** Este laboratorio demuestra que una validación de seguridad colocada en el punto incorrecto del pipeline de procesamiento de datos es completamente inútil si posteriormente la aplicación decodifica de nuevo la entrada.
- **Inseguridad de la Doble Codificación:** Las soluciones que intentan detectar patrones maliciosos en texto plano se vuelven obsoletas cuando el atacante usa *Double URL Encoding*, ya que los mecanismos de inspección solo ven secuencias hexadecimales inocuas mientras que el motor de ejecución final interpreta el payload malicioso.
- **Enfoque en la Arquitectura:** La prevención efectiva de ataques web exige aplicar normalización de datos en el momento de entrada, validación por listas blancas y controles de acceso nativos del sistema de archivos.